

Lecture 3

Priority Queue (Heap) Complexity Analysis



Previous Lecture

Priority Queue ADT: a set of elements, each has a priority (key).

Operations: insert(x,k), find-min(),
delete-min(), decrease-key(x,k)

A possible data structure: Binary heap

An almost complete binary tree whose elements follow the heap property (**min heap**: no node is larger than its children)

A possible implementation of a binary heap: array.

Previous Lecture

Operations on a heap:

- ▷ Percolate up
- ▷ Percolate down

Space:

An array of size N , plus a variable to store the current size n

Time:

- ▷ FindMin: $O(1)$
- ▷ DeleteMin: $O(\log n)$
- ▷ Insert: $O(\log n)$
- ▷ DecreaseKey: $O(\log n)$

Binary Heap Analysis

- ▷ Correctness: Omitted
- ▷ Space:
 - ▷ An array of size N , plus a variable to store the current size n
- ▷ Time
 - ▷ FindMin: $O(1)$
 - ▷ DeleteMin and Insert: $O(\log n)$
 - ▷ DecreaseKey: $O(\log n)$

Binary Heap Analysis

- ▷ Correctness: Omitted
- ▷ Space:
 - ▷ An array of size N , plus a variable to store the current size n
- ▷ Time
 - ▷ FindMin: $O(1)$
 - ▷ DeleteMin and Insert: $O(\log n)$
 - ▷ DecreaseKey: $O(\log n)$
- ▷ What is missing?

Binary Heap Analysis

- ▷ Correctness: Omitted
- ▷ Space:
 - ▷ An array of size N , plus a variable to store the current size n
- ▷ Time
 - ▷ FindMin: $O(1)$
 - ▷ DeleteMin and Insert: $O(\log n)$
 - ▷ DecreaseKey: $O(\log n)$
- ▷ What is missing?
 - ▷ We need to show how to build a heap with N elements

Naïve Build Heap

- ▷ Insert all elements of A into a new Heap array B one by one.
- ▷ Copy the Heap array back into A

NaïveBuildHeap(A, n)

Allocate array B of size n

for $i = 1$ to n

 Insert($B, i, A[i]$)

for $i = 1$ to n

$A[i] = B[i]$

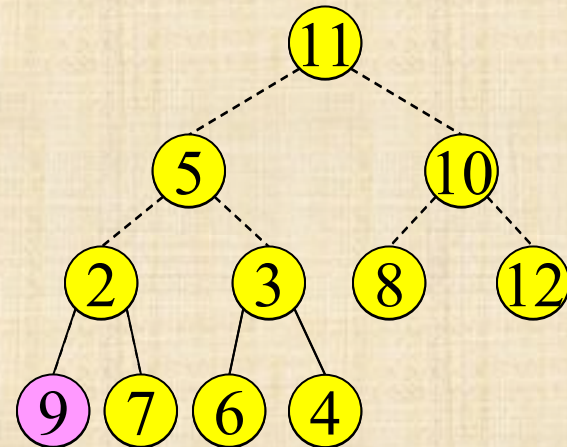
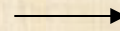
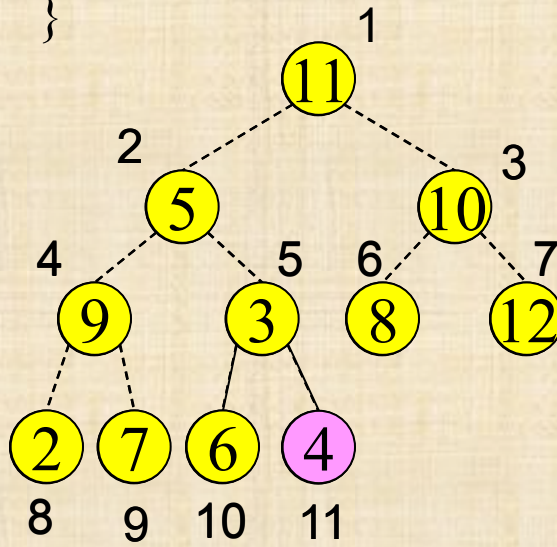
Naïve Build Heap Complexity

- ▷ *NaïveBuildHeap* implementation
 - ▷ n calls to **Insert**. Each invokes **PercUp**.
 - ▷ Time for **PercUp** is proportional to the depth of the current tree (not all elements were inserted yet)
 - ▷ After $\frac{n}{2}$ insertions, depth of tree is $\log \frac{n}{2}$
 - ▷ Time for the last $\frac{n}{2}$ insertions is $\geq \frac{n}{2} \log \frac{n}{2}$
- ▷ We can do better: $O(n)$

Efficient Build Heap bottom up

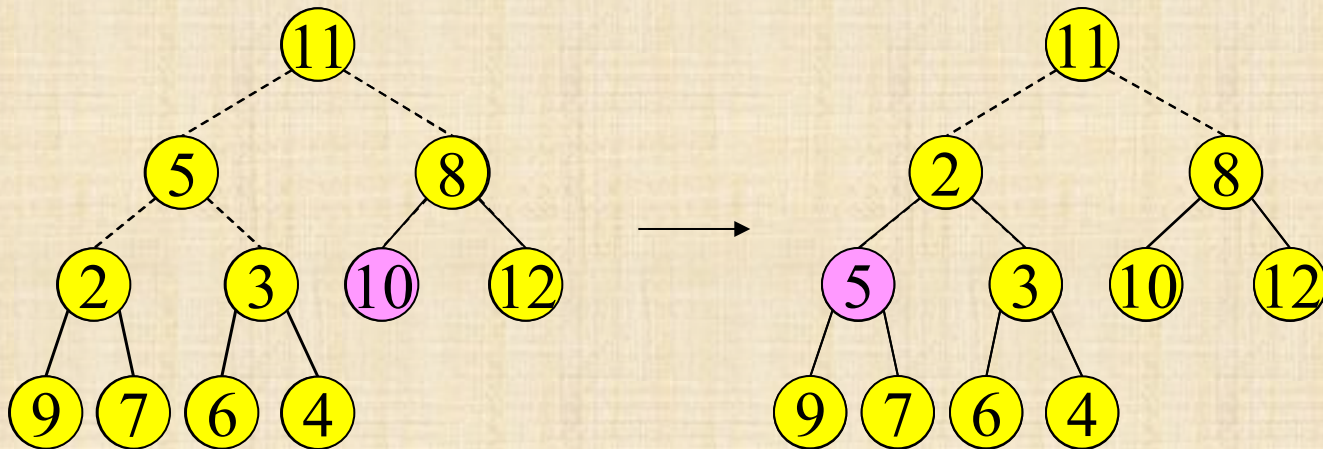
```
BuildHeap (A) {  
   $n \leftarrow \text{length}(A)$   
  for  $i = \lfloor n/2 \rfloor$  down to 1  
    PercDown(i, A[i], n, A)  
}
```

$n=11$

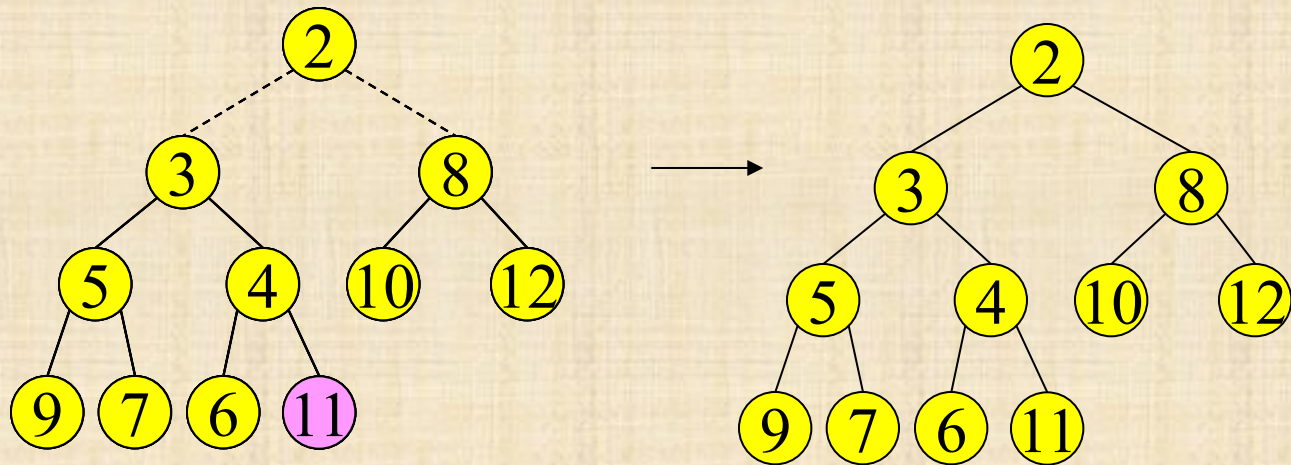


A: array that stores the heap
n: number of elements in
the heap
i: is the index we consider

Build Heap



Build Heap



BuildHeap (A)

$n \leftarrow \text{length}(A)$

for $i = \lfloor n/2 \rfloor$ down to 1

$\text{PercDown}(i, A[i], n, A)$

Correctness

- ▷ We prove by **reverse induction** (we start with n and work our way towards 1) that after handling element $A[i]$, $A[i \dots n]$ satisfies the heap property, that is, no element in $A[i \dots n]$ is larger than its children.
- ▷ **Base:** $i = \lfloor \frac{n}{2} \rfloor + 1$, $A[i \dots n]$ contains only leaves, so $A[i \dots n]$ already satisfies the heap property
- ▷ **Induct. hypothesis:** After handling element $A[i + 1]$, $A[(i + 1) \dots n]$ satisfies the heap property
- ▷ **Step:** For $i \leq \lfloor \frac{n}{2} \rfloor$, by the inductive hypothesis, the only element in $A[i \dots n]$ that might not satisfy the heap property is $A[i]$ itself. Hence, after $\text{PercDown}(i, A[i], n, A)$, the heap property is satisfied for $A[i \dots n]$

Analysis of Build Heap

- ▷ Efficient Build Heap :
 - ▷ Insert all keys into an array in arbitrary order,
then turn it to a heap:
 - ▷ for $i = \lfloor n/2 \rfloor$ downto 1
PercDown($i, A[i], n, A$)
- ▷ How long does it take?

Analysis of Build Heap

▷ **Recall:** If a node has height h , then percolating down the node takes $O(h)$ time

▷ Assume $N = 2^{k+1} - 1$ (a complete tree of height k)

▷ Level 0 (root): k steps for 1 item

▷ Level 1: $k - 1$ steps for 2 items

▷ Level 2: $k - 2$ steps for 4 items

▷ Level i :	$k - i$	steps for	2^i items
---------------	---------	-----------	-------------

▷ Level k : 0 steps for 2^k items

▷ Summing over all levels:

$$\sum_{i=0}^k (k - i)2^i$$

Analysis of Build Heap

$$\sum_{i=0}^k (k-i)2^i =$$

Analysis of Build Heap

$$\sum_{i=0}^k (k-i)2^i = \sum_{i=0}^k i2^{k-i} =$$

Analysis of Build Heap

$$\sum_{i=0}^k (k-i)2^i = \sum_{i=0}^k i2^{k-i} = 2^k \sum_{i=0}^k i2^{-i} \leq$$

Analysis of Build Heap

$$\sum_{i=0}^k (k-i)2^i = \sum_{i=0}^k i2^{k-i} = 2^k \sum_{i=0}^k i2^{-i} \leq N \sum_{i=0}^k i2^{-i} \leq$$

Analysis of Build Heap

$$\sum_{i=0}^k (k-i)2^i = \sum_{i=0}^k i2^{k-i} = 2^k \sum_{i=0}^k i2^{-i} \leq N \sum_{i=0}^k i2^{-i} \leq$$

$$\leq N \sum_{i=0}^k \left(\frac{3}{2}\right)^i 2^{-i} =$$

Analysis of Build Heap

$$\sum_{i=0}^k (k-i)2^i = \sum_{i=0}^k i2^{k-i} = 2^k \sum_{i=0}^k i2^{-i} \leq N \sum_{i=0}^k i2^{-i} \leq$$

$$\leq N \sum_{i=0}^k \left(\frac{3}{2}\right)^i 2^{-i} = N \sum_{i=0}^k \left(\frac{3}{4}\right)^i \leq$$

Analysis of Build Heap

$$\sum_{i=0}^k (k-i)2^i = \sum_{i=0}^k i2^{k-i} = 2^k \sum_{i=0}^k i2^{-i} \leq N \sum_{i=0}^k i2^{-i} \leq$$

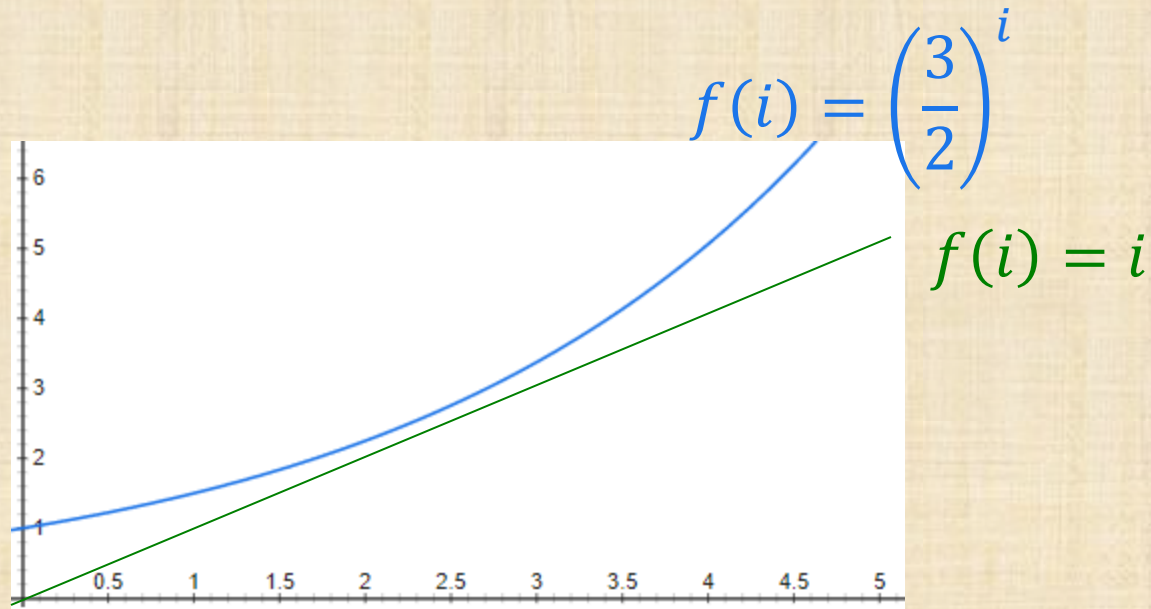
$$\leq N \sum_{i=0}^k \left(\frac{3}{2}\right)^i 2^{-i} = N \sum_{i=0}^k \left(\frac{3}{4}\right)^i \leq 4 \cdot N =$$

Analysis of Build Heap

$$\begin{aligned}\sum_{i=0}^k (k-i)2^i &= \sum_{i=0}^k i2^{k-i} = 2^k \sum_{i=0}^k i2^{-i} \leq N \sum_{i=0}^k i2^{-i} \leq \\ &\leq N \sum_{i=0}^k \left(\frac{3}{2}\right)^i 2^{-i} = N \sum_{i=0}^k \left(\frac{3}{4}\right)^i \leq 4 \cdot N = O(N)\end{aligned}$$

Analysis of Build Heap

▷ for every $i \geq 0$, it holds that $i \leq \left(\frac{3}{2}\right)^i$

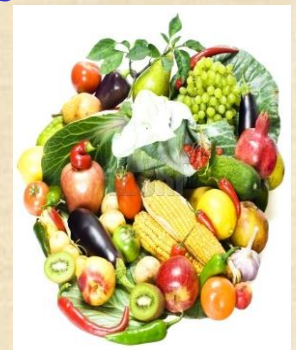


Summary

- ▷ Linear order: sequence, linked list, queue, stack
- ▷ hierarchical order: trees
- ▷ Heap: priority queue
 - ▷ FindMin: $O(1)$
 - ▷ DeleteMin and Insert: $O(\log n)$
 - ▷ DecreaseKey: $O(\log n)$
 - ▷ BuildingHeap: $O(n)$



Trees



Binary Heaps

Complexity for real!

- ▷ Complexity analysis:
 - ▷ Asymptotic bounds:
 - ▷ $O(n)$
 - ▷ $\Omega(n)$
 - ▷ $\Theta(n)$
 - ▷ Best / worst / average case analysis

Evaluating an Algorithm

Steve: My algorithm can sort 10^6 numbers in 3 seconds

Bill: My algorithm can sort 10^6 numbers in 5 seconds

Steve: I've just tested it on my new computer

Bill: I remember my result from my undergraduate studies (1973).

Steve: My input is a random permutation of $1..10^6$

Bill: My input is assumed to be sorted, so I only had to verify that it is sorted

Observations

- ▷ Processing time is not a good measure!!!
- ▷ We need a 'robust' measure, independent of the machine
- ▷ Prior knowledge about the data can affect the time complexity

The word-RAM Model of Computation

- ▷ RAM: Random Access Machine
 - ▷ Each number / pointer is stored in 1 word.
 - ▷ A simple operation takes 1 time step
 - ▷ A memory access takes 1 time step
 - ▷ **Note:** Loops and subroutines are not simple operations
- ▷ For a given problem instance:
Running time of an algorithm = # of steps
- ▷ **Why?** Useful abstraction $\Rightarrow$ allows machine-independent analysis of algorithms

Time Complexity $T(n)$ aka running time

- ▷ $T: \mathbb{N} \rightarrow \mathbb{N}$ a function from Input size to number of steps
- ▷ $T(n)$ is the **maximum** number of steps the algorithm performs on any input of size n
- ▷ **Remark:** we can also define such functions for the **minimal** or **average** number of steps – to be discussed and defined formally later.

Formal definition: Running Time

- ▷ Let A be an algorithm
- ▷ Let x be an input whose length is $|x|$
- ▷ Define $T_{A,x}$ as the running time of A on input x :

$T_{A,x}$ = # of steps that A performs when run on x

- ▷ $T_{A,x}$ depends on A and on x

Formal definition: Running Time

Let $I_{A,n} = \{x: |x| = n\}$ denote the set of all possible inputs of length n to algorithm A

▷ Best case:

$$T_{best}(n) = \min \{ T_{A,x} : x \in I_{A,n} \}$$

▷ Worst case:

$$T_{worst}(n) = \max \{ T_{A,x} : x \in I_{A,n} \}$$

▷ Under the assumption that all $x \in I_{A,n}$ have the same probability

$$\text{Average case: } T_{average}(n) = \frac{1}{|I_{A,n}|} \sum_{x \in I_{A,n}} T_{A,x}$$

Comparing Time Complexities

- ▷ We need a method to compare different time complexity functions
- ▷ We define something even stronger:
 - ▷ Comparison of ANY functions
- ▷ But it is not as simple as it seems!!!

Scope of interest - domain



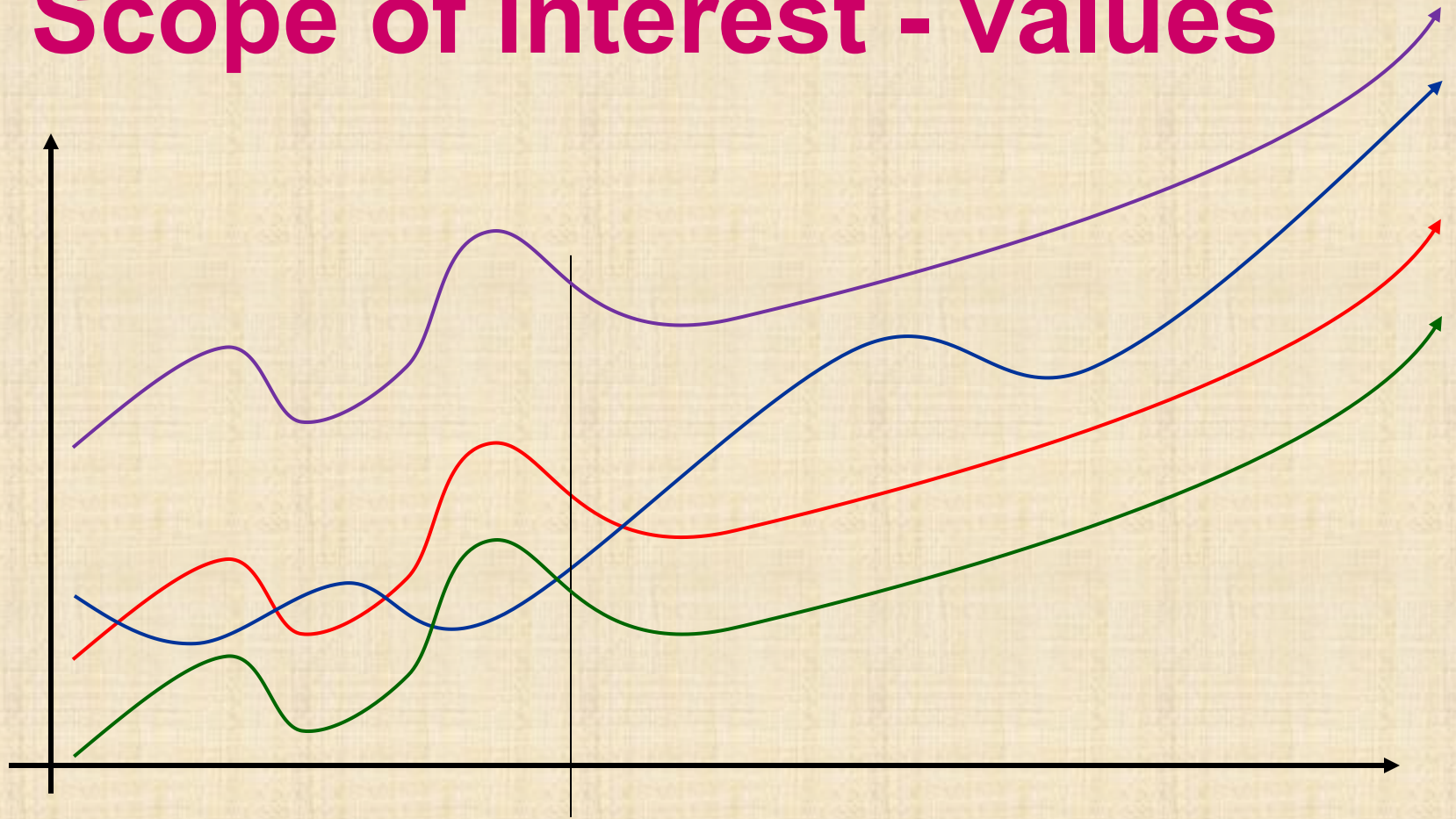
Where I can write a rule for infinite number of x 's

We call this *Asymptotic* behavior

Are there cases where we do not care for

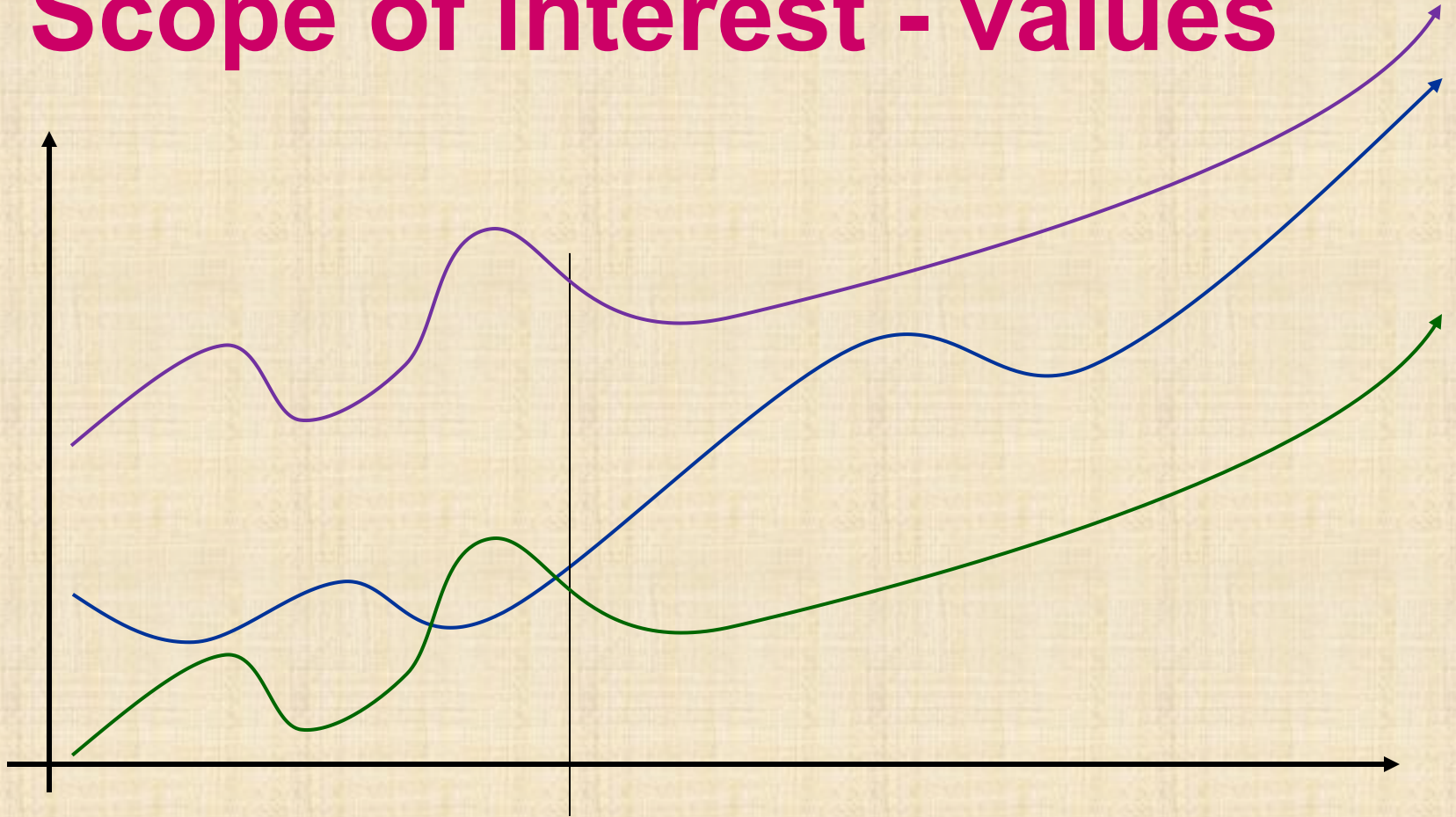
Asymptotic behavior? Yes! (depending on the prob.)

Scope of interest - values



Bounding by constant

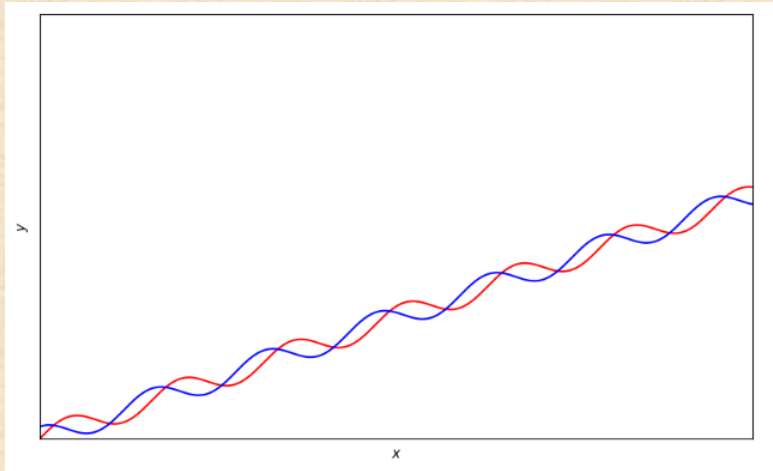
Scope of interest - values



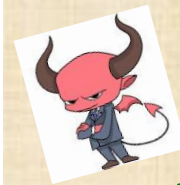
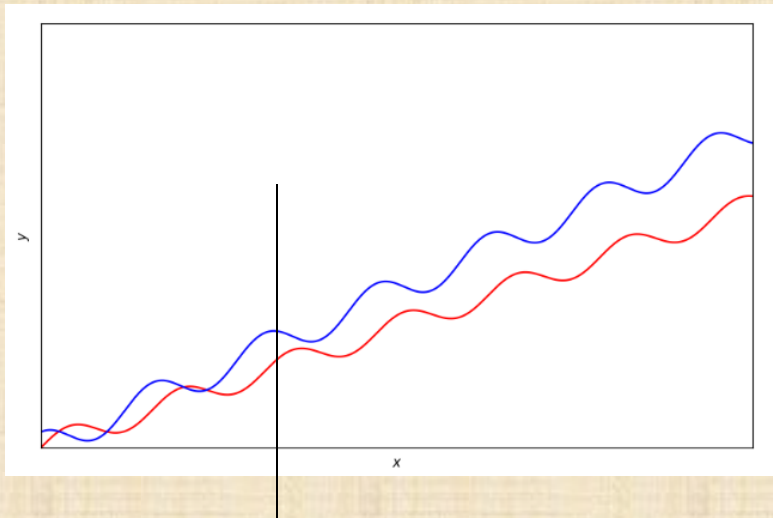
Bounding by adding a constant

Not good enough for a computer model

Scope of interest - values

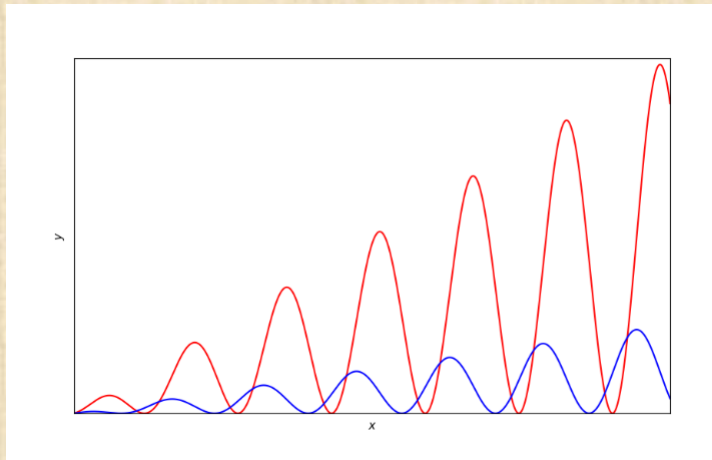
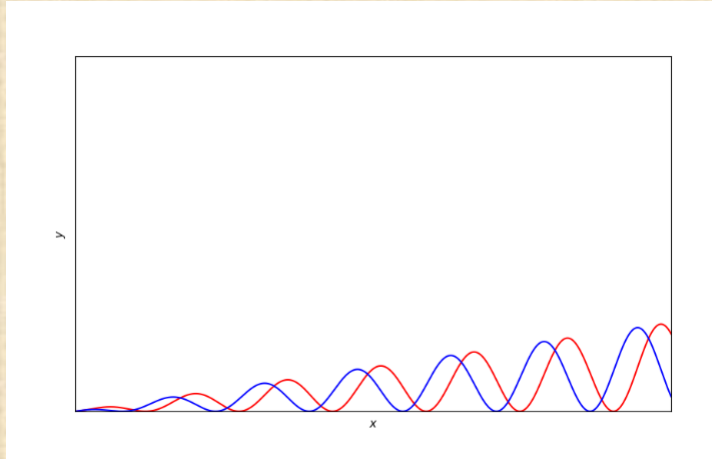


Bounding by
multiplying by a constant



:You're cheating -
now you prove whatever
you want...

Scope of interest - values



Bounding by

multiplying by a constant



: DOES NOT ALWAYS WORK!!

Big-O (Upper Bound)

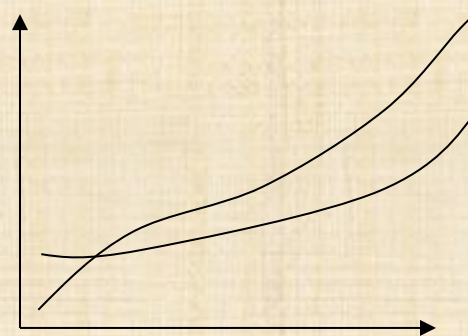
Definition:

$f(n) \in O(g(n))$ if there exist positive constants c and n_0 such that:

for all $n > n_0$, $f(n) \leq cg(n)$

Consider large inputs
(asymptotic behavior)

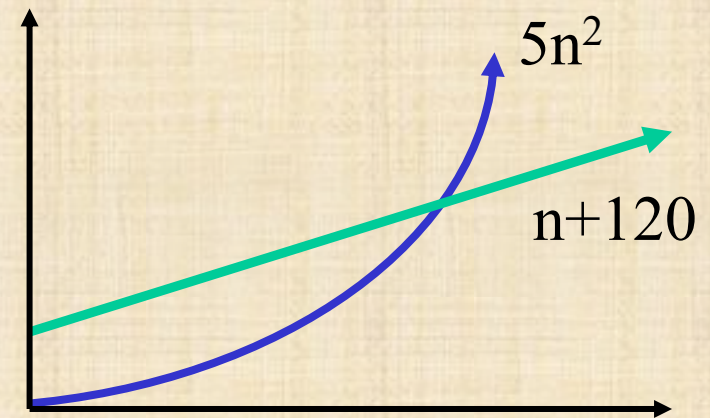
Ignore constants



Order Notation

- ▷ **Notation:** Often, we write $f(n) = O(g(n))$ instead of $f(n) \in O(g(n))$.
- ▷ Order notation is not symmetric:
 $2n^3 + n \in O(n^4)$ does **not** imply that
 $n^4 \in O(2n^3 + n)$
- ▷ The right-hand side is a “rough” upper bound of the left.

Big-O: example



Claim: $n + 120 \in O(n^2)$

Proof: We have to show that there exist $c > 0$ and n_0 , s.t. $\forall n \geq n_0, n + 120 \leq cn^2$

We show that the claim holds for $c = 6$ and $n_0 = 5$

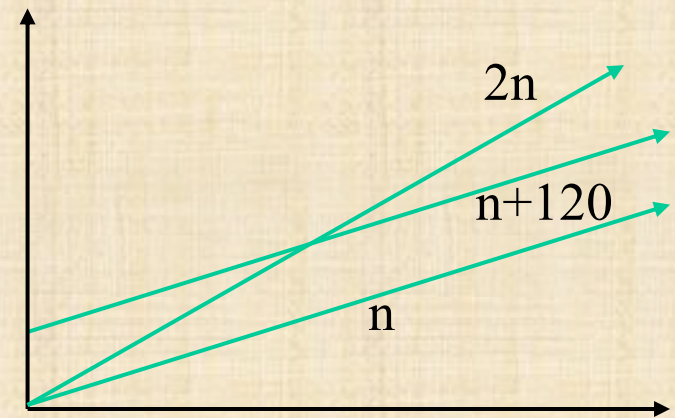
▷ $n \leq n^2$ for $\forall n \geq 1$

▷ $120 < 125 \leq 5n^2$ for $\forall n \geq 5$

▷ Hence $n + 120 \leq n^2 + 5n^2 = 6n^2, \quad \forall n \geq n_0 = 5$ ■

▷ *Note:* We could select other pairs of c, n_0 .
For example, $c=20$ and $n_0=2$.

Big-O: example



Claim: $n + 120 \in O(n)$

Proof: We have to show that there exists $c > 0$ and n_0 , s.t. $\forall n \geq n_0, n + 120 \leq cn$

▷ We show that the claim holds for $c = 2$ and $n_0 = 120$

▷ $\forall n \geq n_0 = 120 : n + 120 \leq n + n = 2n = cn.$

▷ Again, we could select other pairs of c, n_0 . ■

For example, $c=13$ and $n_0=10$ ($10+120 \leq 13 \cdot 10$).

Ω , Θ Notation

(Lower & Tight bounds)

Definition:

$f(n) \in \Omega(g(n))$ if there exists positive constants c and n_0 , s.t. $\forall n \geq n_0, f(n) \geq c \cdot g(n)$

Definition:

$f(n) \in \Theta(g(n))$ if $f(n) \in O(g(n))$ and $f(n) \in \Omega(g(n))$

That is, $\exists c_1 > 0, \exists c_2 > 0, \exists n_0$, s.t. $\forall n \geq n_0$,
$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

Example

Consider $T(n) = 3n^2 + 2n^3$

- ▷ It holds that $T(n) \in O(n^5)$ and $T(n) \in O(2^n)$ and $T(n) \in O(n^3)$.
- ▷ It also holds that $T(n) \in \Omega(1)$ and $T(n) \in \Omega(n^2 \log n)$ and $T(n) \in \Omega(n^3)$

$$T(n) \in \Theta(n^3)$$

Example

Claim: $\frac{1}{2} n^2 - 3n = \Theta(n^2)$

Proof: We have to show that there exist

$c_1, c_2 > 0$ and n_0 , s.t. $c_1 n^2 \leq \frac{1}{2} n^2 - 3n \leq c_2 n^2$

$\forall n \geq n_0$

$$\frac{1}{2} n^2 - 3n \leq \frac{1}{2} n^2, \quad \forall n \geq 1$$

Since $\frac{1}{4}n > 3, \forall n \geq 12$

$$\frac{1}{2} n^2 - 3n = n \left(\frac{1}{2} n - 3 \right) \geq n \left(\frac{1}{2} n - \frac{1}{4} n \right) \geq \frac{1}{4} n^2 ,$$

$$c_2 = \frac{1}{2}, c_1 = \frac{1}{4}, n_0 = 12$$

QED

Example

- ▷ Claim: $6n^3 \notin \Theta(n^2)$
- ▷ Proof (by contradiction):
 - ▷ Assume $\exists c_1, c_2 > 0$ and $\exists n_0$, s.t. $\forall n \geq n_0$ it holds that $c_1 n^2 \leq 6n^3 \leq c_2 n^2$
 - ▷ It follows that $n \leq \frac{c_2}{6}$,
but we assume that the inequality holds $\forall n \geq n_0$.

A contradiction

- ▷ Note $6n^3 \in \Omega(n^2)$

Big-O Algebra

▷ Given:

▷ $T_1(n) = O(f(n))$

▷ $T_2(n) = O(g(n))$

▷ It holds that

1. $T_1(n) + T_2(n) \in O(\max\{f(n), g(n)\})$

2. $T_1(n) \cdot T_2(n) \in O(f(n) \cdot g(n))$

Proof of 1: $T_1(n) + T_2(n) \in O(\max\{f(n), g(n)\})$

- ▷ We have to show that $\exists c$ and $\exists n_0$ s.t.,
 $\forall n \geq n_0, T_1(n) + T_2(n) \leq c \max\{f(n), g(n)\}$
- ▷ $T_1(n) \in O(f(n))$ implies $\exists c_1$ and n_1 s.t.,
 $\forall n \geq n_1, T_1(n) \leq c_1 f(n)$
- ▷ $T_2(n) \in O(g(n))$ implies $\exists c_2$ and n_2 s.t.,
 $\forall n \geq n_2, T_2(n) \leq c_2 g(n)$
- ▷ Let $n_0 = \max\{n_1, n_2\}$. It follows that $\forall n \geq n_0$
$$T_1(n) + T_2(n) \leq c_1 f(n) + c_2 g(n)$$
$$\leq c \max\{f(n), g(n)\}$$

for $c = c_1 + c_2$

Proof of 2: $T_1(n) \cdot T_2(n) \in O(f(n) \cdot g(n))$

- ▷ We have to show that $\exists c$ and $\exists n_0$ s.t.,
 $\forall n \geq n_0, T_1(n) \cdot T_2(n) \leq c f(n) \cdot g(n)$
- ▷ $T_1(n) \in O(f(n))$ implies $\exists c_1$ and n_1 s.t.,
 $\forall n \geq n_1, T_1(n) \leq c_1 f(n)$
- ▷ $T_2(n) \in O(g(n))$ implies $\exists c_2$ and n_2 s.t.,
 $\forall n \geq n_2, T_2(n) \leq c_2 g(n)$
- ▷ Let $n_0 = \max\{n_1, n_2\}$. It follows that $\forall n \geq n_0$
$$T_1(n) \cdot T_2(n) \leq c_1 \cdot c_2 \cdot f(n) \cdot g(n)$$
$$\leq c \cdot f(n) \cdot g(n)$$

for $c = c_1 \cdot c_2$

More Examples

▷ $\log_2 n^{10} \in O(\log_5 n)$

▷ $16 \cdot 2^n \in O(2^{2n})$

Big-O and polynomials

- ▷ A polynomial of degree k is $O(n^k)$

Proof:

- ▷ Let $f(n) = b_k n^k + b_{k-1} n^{k-1} + \dots + b_1 n + b_0$

- ▷ Let $a = \max_i \{b_i\}$

- ▷
$$\begin{aligned} f(n) &\leq a n^k + a n^{k-1} + \dots + a n + a \\ &\leq (k+1) a n^k \leq c n^k \quad \text{for } c = (k+1)a \end{aligned}$$

More Examples

- ▷ $n(n - 1) \in \Omega(n^2)$
- ▷ $123456 \in O(1)$
- ▷ $4x^2 + 100 \in O(x^2)$
- ▷ $4x^2 + 100 \in \Omega(x^2)$
- ▷ $4x^2 + 100 \in O(x^3)$
- ▷ $4x^2 + 100 \in \Theta(x^2)$
- ▷ $4x^2 + 100 \notin \Theta(x^3)$
- ▷ $4x^2 - 100 \in O(x^2)$
- ▷ $4x^2 + x \cdot \log x \in O(x^2)$
- ▷ $16n^3 \log_8(10n^2) + 100n^2 \in O(n^3 \log(n))$

Not in class

Some Basic Terminology

- ▷ Constant time: $O(1)$
- ▷ Logarithmic time: $O(\log n)$
- ▷ Linear time: $O(n)$
- ▷ Quadratic time: $O(n^2)$
- ▷ Cubic time: $O(n^3)$
- ▷ Polynomial time: $O(n^k)$ for some k
- ▷ Exponential time: $O(c^n)$ for some $c > 1$

Many Types of Analysis

- ▷ Various cases:
 - ▷ Worst-case / Best-case: over all inputs of length n
 - ▷ Average case: assumes a distribution of inputs
- ▷ Various bound:
 - ▷ Upper bound ($O(\cdot)$) vs. lower bound ($\Omega(\cdot)$)
 - ▷ Tight bound ($\Theta(\cdot)$)
- ▷ All possible combinations are valid

Types of Analysis - example

- ▷ Given a list of n numbers (not sorted) find whether a number x appears in the list
- ▷ Algorithm: check elements one after the other
 - ▷ Worst case: $O(n^2)$, $\Omega(1)$, $\Theta(n)$, $O(n)$, $\Omega(n)$
 - ▷ Best case: $O(\sqrt{n})$, $\Omega(1)$, $\Theta(1)$, $O(1)$
 - ▷ Average case: $O(n^{1.0})$, $\Omega(\log n)$, $\Theta(n)$, $O(n)$, $\Omega(n)$

Example: Sorting



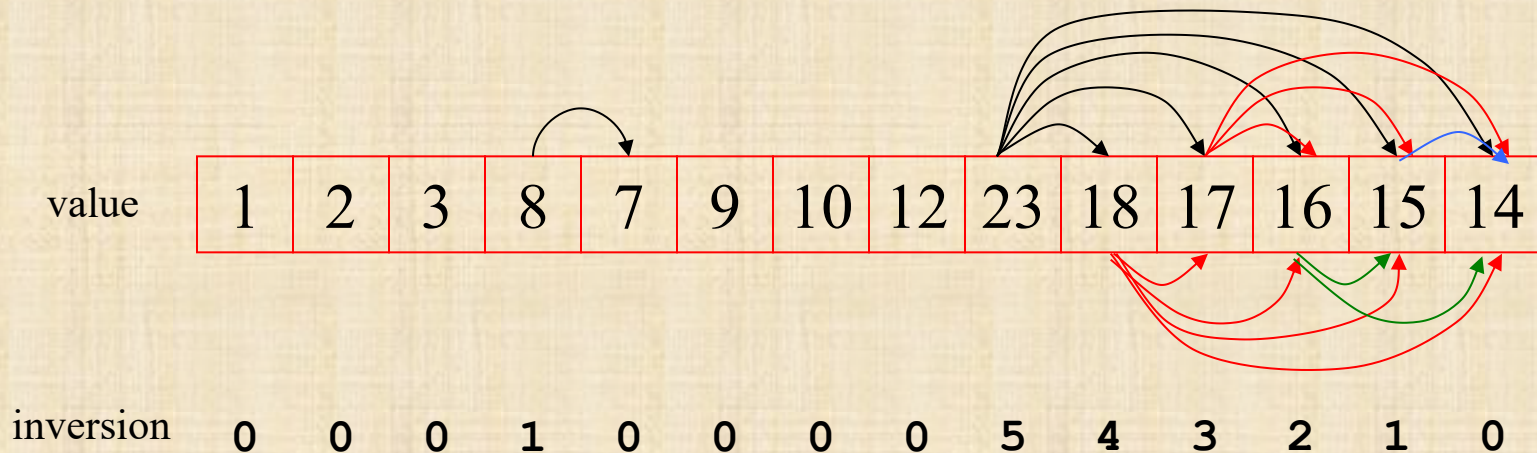
- ▷ Input:
 - ▷ An array A of data records (e.g. integers)
 - ▷ A comparison function which imposes a total order on the records
- ▷ Output:
 - ▷ Reorder the elements of A such that for any i and j , if $i < j$ then $A[i] \leq A[j]$

Inversions

- ▷ An inversion is a pair of elements in wrong order: $i < j$ but $A[i] > A[j]$
- ▷ By definition, a sorted array has no inversions
- ▷ Sorting can be regarded as the process of removing all inversions

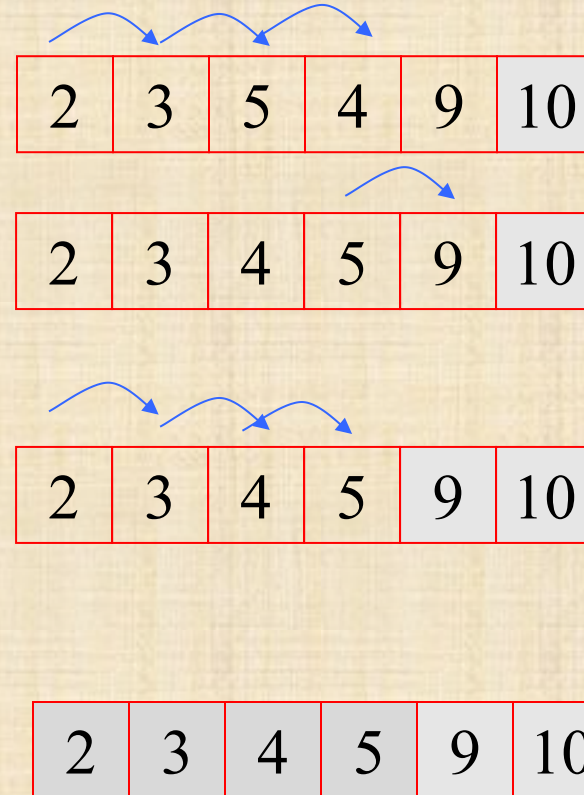
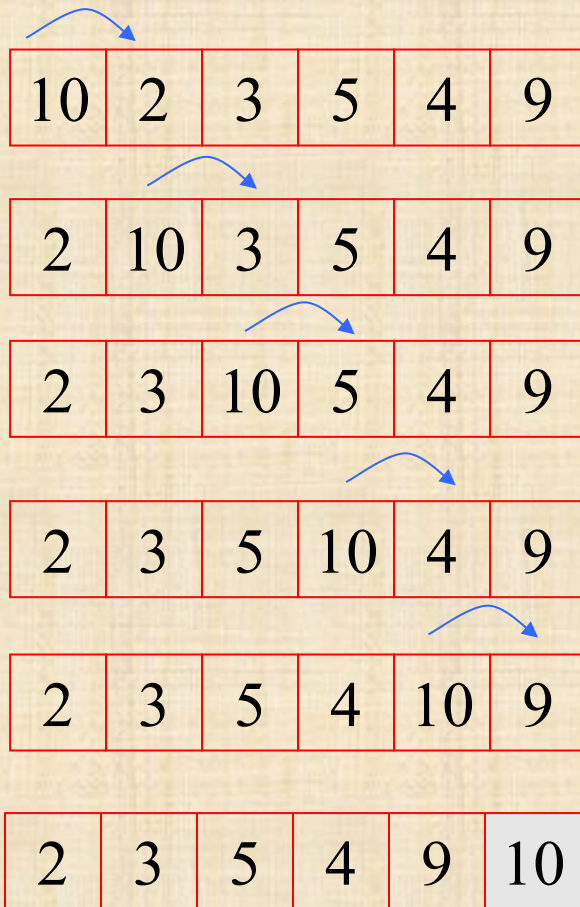
Inversions

- ▶ A single value out of place can cause several inversions



Bubble sort: A sorting algorithm based on removal of inversions of neighboring elements.

Bubble Sort



Bubble Sort

demo

```
BubbleSort(A, n)
  for i = 1 to n-1 do
    swap=0
    for j = 2 to n-i+1 do
      if A[j-1] > A[j] then
        swap=1
        t = A[j-1]
        A[j-1] = A[j]
        A[j] = t
      end
    if swap == 0 then return
  end
```

A[1..n]: an array
n: its size

swap

swap – binary indicator for detecting early termination

Bubble Sort Complexity

- ▷ Worst case - reverse order

- ▷ Number of steps:

$$(n - 1) + (n - 2) + \dots + 1 = \sum_{i=1}^{n-1} i = \frac{(n - 1)n}{2} \in \Theta(n^2)$$

- ▷ Best case - no inversions (only 1 iteration) $\Theta(n)$

Bubble Sort Complexity

▷ Average case

- ▷ In $\frac{1}{2}$ of the inputs the minimum element is in the second half of the array
- ▷ Minimum element only moves one position in each iteration, so $\geq n/2$ iterations
- ▷ # steps for each such input $\geq (n-1) + (n-2) + \dots + \frac{n}{2} = \Omega(n^2)$
- ▷ Since $\frac{1}{2}$ the inputs contribute $\Omega(n^2)$ each, $T_{aver}(n) \in \Omega(n^2)$
- ▷ Since $T_{aver}(n) \leq T_{worst}(n)$ and $T_{aver}(n) \in \Omega(n^2)$ it follows that $T_{aver} \in \Theta(n^2)$

Bubble Sort: Analysis

- ▷ Time complexity:
 - ▷ Worst case: $\Theta(n^2)$
 - ▷ Best case: $\Theta(n)$
 - ▷ Average: $\Theta(n^2)$

Analyzing Recursive Programs

Example: Binary Search



Binary Search

Call *BSearch*(*x*, *A*, 0, *n*-1) to get the result of binary search

```
function BSearch(x, A, i, j)
if (j-i < 0) return -1;    % not in A
m ← ⌊(i+j)/ 2⌋;
if (x = A[m]) return m;
if (x < A[m])
    return BSearch(x, A, i, m-1);
else
    return BSearch(x, A, m+1, j);
```

A: an array
x: the value we search for
i,j: two indices of A
(the range we search in)

What is the worst-case running time?

$\Theta(\log n)$

Binary Search – writing complexity function

```
function BSearch(x, A, i, j)
if (j-i < 0) return -1;    % not in A
m ← ⌊(i+j)/ 2⌋;
if (x = A[m]) return m;
if (x < A[m])
    return BSearch(x, A, i, m-1);
else
    return BSearch(x, A, m+1, j);
```

A: an array
x: the value we search for
i,j: two indices of A
(the range we search in)

Notation:

b = # steps for base case

c = bound on the # steps to get ready to do a recursive call

n - the size of the sub-problem is $j-i+1$

Running time $T(n)$ satisfies: $T(1) = b$, $T(n) \leq T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c$

Solving Recursive Equations by Repeated Substitution

Consider *BSearch* on input of size n

$$T(n) \leq T\left(\frac{n}{2}\right) + c$$

$$\leq T\left(\frac{n}{4}\right) + c + c = T\left(\frac{n}{4}\right) + 2c$$

$$\leq T\left(\frac{n}{2^3}\right) + c + 2c = T\left(\frac{n}{2^3}\right) + 3c$$

...

$$\leq T\left(\frac{n}{2^j}\right) + jc$$

...

$$\leq T\left(\frac{n}{2^{\log n}}\right) + c \log n$$

$$= T(1) + c \log n = b + c \log n \in O(\log n)$$

not a formal proof

b: number of steps
for base case: $T(1)$
c: bound on number of steps
in each iteration

Solving Recursive Equations by Induction

- ▷ The informal substitution method tells us what the inductive claim should be.

BSearch: $T(n) = O(\log n)$

- ▷ Simplifying assumption: n is a power of 2, $n = 2^k$, $k = \log n$
- ▷ $T(n) \leq T\left(\frac{n}{2}\right) + c$, let $T(1) = b$

Claim: $T(2^k) \leq b + ck$ so $T(n) \leq b + c \log n$

Proof of claim: by induction on k

- ▷ Base case: $k = 0$, $T(2^0) = T(1) = b$
- ▷ Inductive assumption: $T(2^k) \leq b + ck$
- ▷ Induction step (for $k + 1$)
$$T(2^{k+1}) \leq T\left(\frac{2^{k+1}}{2}\right) + c = T(2^k) + c \leq b + c \cdot k + c = b + c \cdot (k + 1)$$

BSearch: $T(n) = O(\log n)$

▷ Complete proof: General n

$$k \leq \log n + 1$$

$$▷ T(n) \leq T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c$$

Claim: if $2^{k-1} \leq n < 2^k$, then $T(n) \leq b + ck$

Proof of claim: by induction on k for $2^{k-1} \leq n < 2^k$

▷ Base case: $k = 1$, $T(2^0) = T(1) = b$

▷ Inductive assumption for $1, \dots, k$: if $2^{k-1} \leq n < 2^k$
then $T(n) \leq b + ck$

▷ Induction step for $k + 1$, let $2^k \leq n < 2^{k+1}$

$$\text{so } 2^{k-1} \leq \left\lfloor \frac{n}{2} \right\rfloor < 2^k \text{ and } 2^{k-2} \leq \left\lfloor \frac{n}{2} \right\rfloor - 1 < 2^k$$

$$T(n) \leq \max\left\{T\left(\left\lfloor \frac{n}{2} \right\rfloor\right), T\left(\left\lfloor \frac{n}{2} \right\rfloor - 1\right)\right\} + c \leq b + ck + c = b + c(k + 1)$$

▷ Since $k \leq \log n + 1$, we get: $T(n) \leq (b + c) + c \log(n + 1)$

Tight Bound – binary search

- ▷ We proved $T(n) \in O(\log n)$
- ▷ We can also show that $T(n) \in \Omega(\log n)$
 - ▷ How?
 - ▷ Choose b' and c' and prove by induction that $T(n) \geq b' + c' \log_2 n$
 - ▷ same structure as the upper bound
- ▷ Corollary: $T(n) \in \Theta(\log n)$

Little-o (Non-Tight Upper Bound)

$f(n) \in o(g(n))$ if for every constant $c > 0$ there exists n_0 such that for all $n > n_0$, $f(n) < cg(n)$

Equivalently, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$

E.g.:

$$2n = o(n^2)$$

$$\log n = o(n)$$

$$\text{but } n \notin o(2n)$$

Little- ω

(Non-Tight Lower Bound)

$f(n) \in \omega(g(n))$ if for every constant $c > 0$ there exists n_0 such that for all $n > n_0$, $f(n) > cg(n)$

Equivalently, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$

E.g. :

$$n \log n = \omega(n),$$

$$2^{0.001n} = \omega(n^{300})$$

Summary

- ▷ Machine independent analysis of the running time of an algorithm on large inputs
 - ▷ “ O ”: upper bound
 - ▷ “ Ω ”: lower bound
 - ▷ “ Θ ”: tight bound
- ▷ Ignoring constant factors allows quantitative comparison in a machine independent way.
- ▷ Cases: best / worst / average

Next 3 Lectures: Sort Algorithms

We've already seen:

- ▷ Insertion sort
- ▷ Bubble sort

▷ Next up:

- ▷ Heap sort
- ▷ Merge sort
- ▷ Quick sort